PROJECT 09

Expense Tracker

Business Case Study

A Python console application for recording income and expenses, organizing transactions, reviewing monthly finances, and retaining records in a CSV file.

Project type	Beginner-friendly Python OOP application
Primary user	An individual who wants a simple way to track personal cash flow
Core output	Transaction history and monthly income, expense, and savings totals
Storage	Local CSV file (transactions.csv)

Important: This case study describes the application's intended business use and uses illustrative example figures only. No real user data or measured business impact was supplied.

1. Executive Summary

Many people receive income from one or more sources and spend money across categories such as food, transport, shopping, bills, and subscriptions. When transactions are recorded inconsistently, it can be difficult to understand where money goes or how much remains at the end of a month.

The Expense Tracker is a menu-driven Python application designed to make this process more organized. Users can record income and expenses, view and search transactions, and generate a monthly summary. Records are saved in a CSV file and loaded when the program starts, allowing the user to retain a basic transaction history between sessions.

2. Business Problem

The problem addressed is not a lack of sophisticated financial software; it is the everyday difficulty of maintaining a consistent, accessible record of personal cash flow. Notes, memory, or scattered records can make monthly review time-consuming and increase the chance that a transaction is overlooked.

• **Limited visibility:** users may not know their total spending for a particular month.

• **Scattered records:** transaction details may be spread across notes or separate files.

• **Manual calculations:** adding income and expenses by hand takes time and can lead to mistakes.

• **Weak categorization:** without categories, it is harder to find recurring spending patterns.

3. Proposed Solution

The application provides a simple console menu with six options: record income, record an expense, view all transactions, generate a monthly summary, search transactions, and exit. Each transaction stores its type, amount, category, description, and date.

Feature	Business value
Record income and expenses	Creates a consistent record of money received and spent.
Categorize transactions	Makes spending easier to review by purpose.
Monthly summary	Shows total income, total expenses, and the difference between them.
Search records	Helps users locate transactions by category or description.
CSV persistence	Keeps records available across program restarts when the file is retained.
Input and file error handling	Provides clearer feedback when inputs or file operations fail.

4. Target Users and Use Case

The initial target user is an individual who wants a lightweight offline tool and is comfortable running a Python script. A typical use case is entering salary or freelance income and then recording daily purchases. At month-end, the user selects the monthly summary option and reviews the calculated totals.

Example workflow

1. Start the application and choose Record Income.

2. Enter the amount, category, description, and date.

3. Choose Record Expense and enter a purchase.

4. Repeat as needed; the application saves records to the CSV file.

5. Choose Monthly Summary and enter the month in YYYY-MM format.

6. Review the totals and search individual records when more detail is needed.

5. Illustrative Financial Example

The following example demonstrates how the monthly summary works. These values are fictional and are not results measured from a real user.

Metric	Illustrative amount
Monthly income	■28,000
Food expenses	■1,200
Transport expenses	■500
Total expenses	■1,700
Remaining amount (income – expenses)	■26,300

Calculation: ■28,000 – ■1,700 = ■26,300. The application labels this difference as savings; it is a simple cash-flow calculation and does not account for debts, investments, or unrecorded transactions.

6. Technical Design

The project uses four classes:

Class	Responsibility
Transaction	Abstract base class containing shared transaction attributes and methods.
Income	Inherits from Transaction and identifies a record as income.
Expense	Inherits from Transaction and identifies a record as an expense.
ExpenseTracker	Manages the transaction list, CSV persistence, searching, and summaries.

The project demonstrates encapsulation through private transaction attributes and getter/setter methods; inheritance through Income and Expense; abstraction through an ABC base class; file handling through CSV; and exception handling for invalid values and file-related errors.

7. Success Measures

The current project is a learning prototype, so no real-world impact metrics have been measured. Its practical quality can be assessed using the following checks:

• A valid income or expense can be added and appears in the transaction list.

• Records remain available after closing and restarting the program.

• The monthly summary matches manually calculated totals for a test month.

• Search returns matching category or description text.

• Invalid amounts and malformed dates prompt the user to correct the input.

• A missing or unreadable CSV file produces an understandable error message.

8. Risks and Limitations

Risk / limitation	Implication and mitigation
Manual data entry	Missed transactions can make summaries inaccurate; enter records consistently and review them.
Local CSV storage	The file can be lost or edited; keep backups and protect access to the computer.
No authentication or encryption	Not suitable for sensitive financial records or shared computers without additional safeguards.
No editing or deletion feature	Mistakes are harder to correct; add controlled edit/delete functionality in a future version.
Simple savings calculation	Only recorded income and expenses are included; it is not a full accounting or budgeting system.

9. Future Enhancements

• Add the ability to edit and delete transactions safely.

• Provide category-wise expense totals and budget limits.

• Add charts to show spending patterns over time.

• Export monthly reports to PDF or Excel.

• Build a graphical user interface for non-technical users.

• Add automated tests and stronger data validation.

10. Conclusion

The Expense Tracker demonstrates how a small Python application can address a practical personal-finance record-keeping need. Its menu-driven workflow, reusable classes, CSV persistence, and monthly calculations make it a useful beginner portfolio project. With stronger validation, record correction, privacy safeguards, and richer reporting, it could be developed into a more capable personal budgeting tool.

Prepared by: Shriya Verma

Project 09 — Python Learning Portfolio